You can `Control-F` a document…

You can't `Control-F` a video…

ASKABLE

Make any video askable.

Paste a YouTube link. Get a source you can interrogate in plain language — where **every answer cites the exact second it came from**, as a link that jumps you there.

BUILT WITH NATIVE.BUILDER

REACT · VITE · TYPESCRIPT

SUPABASE EDGE FUNCTIONS

RAG OVER PGVECTOR

A 90-minute video is the worst database ever built.

01 The answer **is in there** — buried somewhere between minute 12 and minute 74.

02 Search gives you **the video, not the moment**. You still have to watch it.

03 Auto-captions are a wall of text with **no structure and no citations**.

04 So people skim at 2× and **quote from memory** — which is how misquotes happen.

THREE STEPS, ONE PASTE

From a link to a queryable source.

STEP 01

Paste a link

A YouTube URL, or upload an audio/video file directly. Nothing to install, nothing to configure.

STEP 02

It transcribes and indexes

The transcript is chunked **with its timings intact**, embedded at 1536 dimensions, and stored in Postgres with pgvector. Progress streams back live.

STEP 03

Ask in plain language

Answers come back grounded in the transcript, each one carrying the timestamps it was built from — clickable, and pointing at the exact second.

One complete workflow, end to end: submit → transcribe → embed → retrieve → answer → verify.

SAMPLE SESSION · ILLUSTRATIVE

What did they say about the inflation target?

They describe the target as unchanged, and spend most of the segment on **why the path back to it is slower than expected** — pointing at services pricing rather than energy as the reason.

04:12 11:38 27:05

How many times do they say "recession"?

Three times — at 08:41, 22:16 and 55:03, each with the surrounding sentence quoted.

VERIFIABLE BY CONSTRUCTION

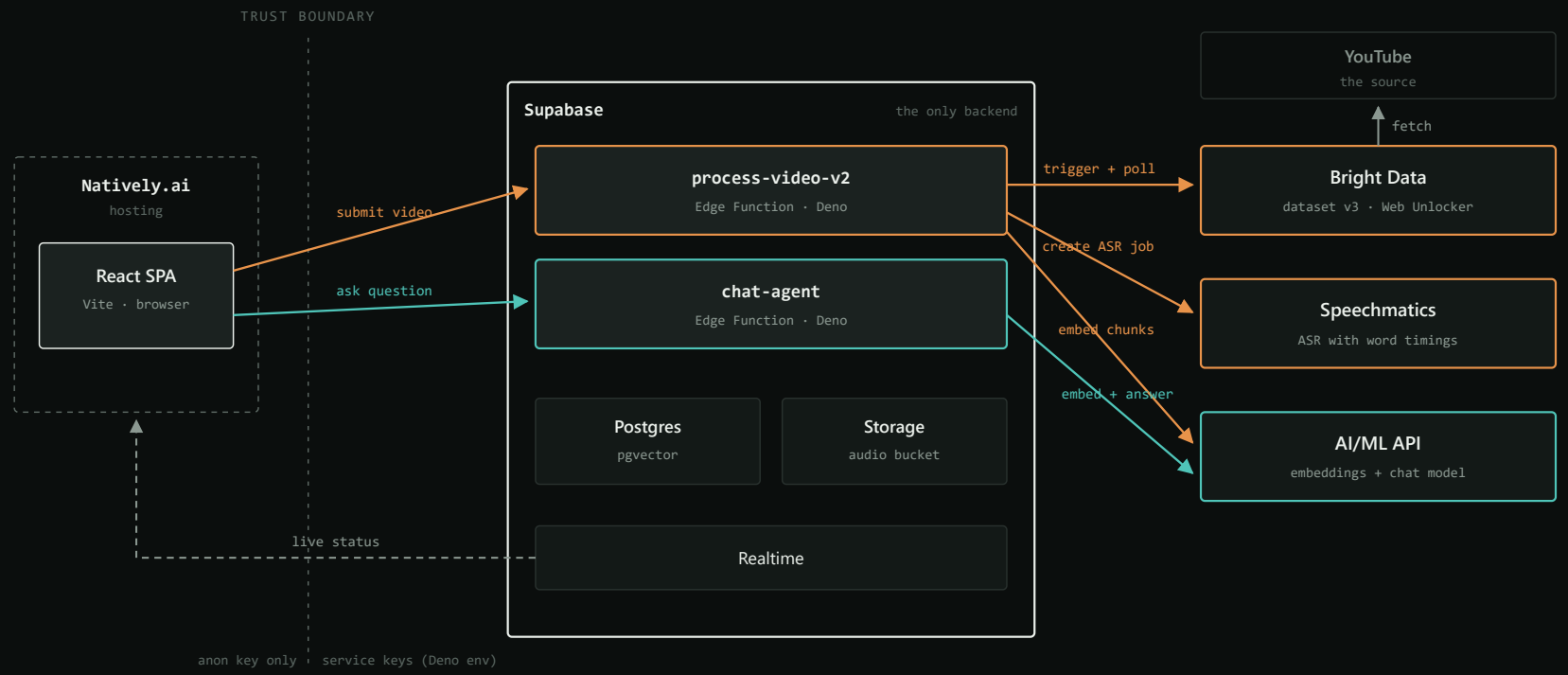
Every claim ships with the second it came from.

Click a timecode and the video opens at that moment. The answer is not something to trust — it is something to **check**, in one click.

Counting questions bypass retrieval entirely and scan every segment, so the count is exact rather than whatever the top-k window happened to contain.

APPLICATION OF TECHNOLOGY

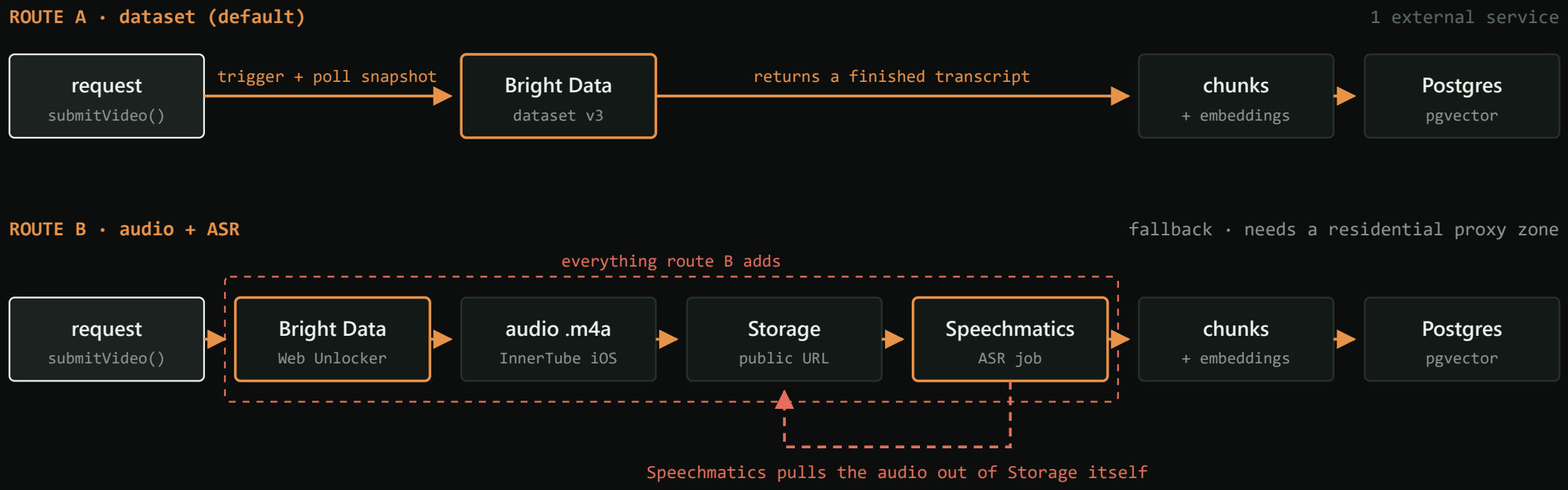
One backend. Three specialists behind it.



capture path question path push · no credential of its own The browser never holds a third-party key.

ORIGINALITY · RESILIENCE

Two independent ways in.



The default route asks Bright Data for a finished transcript — one hop, no proxy, no audio. When that isn't available, the same function walks the long way: proxy the player, download the audio, park it in Storage, and hand Speechmatics a **URL rather than a file**.

Collections can outlive a single invocation, so a job that doesn't finish in time is **parked, not lost** — the row keeps its snapshot id and a resume call picks it up exactly where it stopped.

THE INSIGHT THAT MAKES IT TRUSTWORTHY

Retrieval can't count.



Ask **"how many times do they say X"** and top-k retrieval will confidently undercount — it only ever saw eight chunks. So that question skips retrieval entirely and reads the whole transcript.

And a literal miss is not proof of absence: ASR writes names phonetically, so a failed word search **falls through to semantic search** instead of answering "never mentioned" about a video that discusses it at length.

BUILT WITHOUT LIMITS

Described in plain language. Shipped as a production app.

All of Askable — frontend, edge functions, database schema, vector search, the third-party integrations — was built with **Native.builder**. No boilerplate, no scaffolding by hand: the agents planned, wrote and shipped a real React codebase, and the work went into the parts that actually needed judgment.

Frontend	Backend	Data	Integrations
React + Vite + TypeScript. Submission, live status, chat panel, transcript sources.	**Supabase Edge Functions** on Deno — one for ingestion, one for the chat agent.	**Postgres + pgvector** for chunks and embeddings, Storage for audio, Realtime for status.	**Bright Data, Speechmatics, AI/ML API** — every key held server-side only.

This deck and the architecture behind it were derived from the shipped code, not written ahead of it.

BUSINESS VALUE

Anyone who has to quote a video correctly.

- → **Researchers and students**
  Turn a lecture series into a searchable, citable source.
- → **Journalists and analysts**
  Find the exact sentence in a two-hour press conference — with the link that proves it.
- → **Product and support teams**
  Mine recorded calls and demos without rewatching them.
- → **Creators**
  Pull clips, quotes and chapter markers out of their own back catalogue.

A video archive becomes a knowledge base you can ask questions of.

Everything is already per-video and multi-tenant by construction, so the next steps are additive rather than a rewrite: **search across a whole channel**, playlist ingestion, and shared team workspaces.

LIVE NOW

Stop scrubbing. Start asking.

Paste a link. Ask it something. **Askable** — make any video askable.

Open the live app →

Source code